Desarrollo Web en Entorno Cliente

Introducción a Angular

Índice

Esquema 3

Material de estudio 4

4.1. Introducción y objetivos 4

4.2. Partes de una aplicación de angular 5

4.3. Standalone components vs módulos 7

4.4. ¿Qué es un componente? 9

4.5. Cómo crear y partes de un componente 10

4.6. Componentes principal y componentes hijos. ¿Cómo se enlazan? 15

4.7. Funciones de ciclo de vida de un componente 18

4.8. Hojas de estilos de componente 22

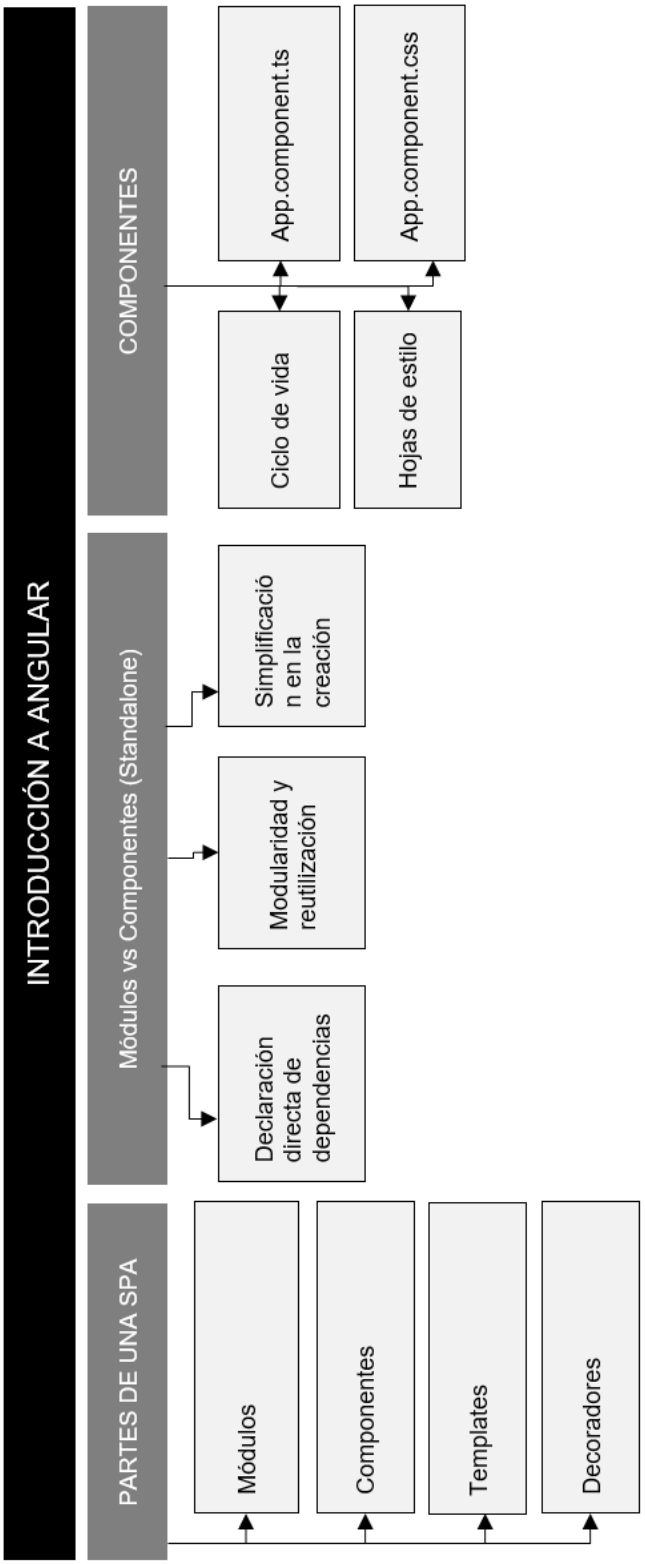
4.9. Referencias bibliográficas 25

A fondo 26

Entrenamientos 27

Test 30

Esquema


INTRODUCCIÓN A ANGULAR
PARTES DE UNA SPA
Módulos
Componentes
Templates
Decoradores
Módulos vs Componentes (Standalone)
Declaración directa de dependencias
Modularidad y reutilización
Simplificación en la creación
COMPONENTES
Ciclo de vida
Hojas de estilo
App.component.ts
App.component.css

Material de estudio

4.1. Introducción y objetivos

A partir de este tema vamos a empezar a descubrir cómo podemos generar aplicaciones web **front-end** a través de una de los frameworks JavaScript más usados del momento. Angular se ha convertido en uno de los frameworks más populares para el desarrollo de SPA (Single Page Applications).

Existen dos versiones de Angular: **Angular 1** o como se conoce AngularJS y **Angular 2** y sucesivas versiones (actualmente nos encontramos en la 18, el momento de escribir estas líneas). Son mantenidos en gran parte por Google, aparte de una gran comunidad de desarrolladores que aportan sus modificaciones para mejorar el framework.

Nosotros vamos a basar el contenido de este módulo en Angular 2 y sucesivas, ya que AngularJS fue una primera versión que sufrió un cambio muy complejo y que nada tiene que ver con las versiones actuales de Angular. Aquí deberéis tener especial cuidado con buscar información de framework y revisar cuando se esté hablando de Angular 1 (AngularJS) o Angular 2 y sucesivos, que las cosas cambian mucho.

El objetivo de este módulo es adquirir las herramientas necesarias para trabajar con Angular en todas sus versiones desde la 2 en adelante.
Durante el desarrollo de este tema se deben conseguir los siguientes objetivos:

- Conocer que es una SPA.
- Conocer las características de los modulos.
- Conocer las características de los componentes.
- Conocer y manejar la estructura de componentes Standalone.
- Manejar correctamente las hojas de estilo de los componentes.

4.2. Partes de una aplicación de angular

Angular es un framework front-end basado en JavaScript y Typescript. Se usa para el desarrollo de aplicaciones web multiplataforma y está pensado para desarrollar aplicaciones del tipo SPA (Single Page Application).

¿Y qué es un SPA? Traducido del inglés sería, una aplicación que ejecuta su contenido en una sola página, pero realmente no es el termino exacto, sería una aplicación web o página web que interactúa con el usuario reescribiendo dinámicamente la página con los nuevos datos que le lleguen del servidor web, esto permite que la carga del contenido sea más rápida y la experiencia de usuario más satisfactoria en cuanto a rendimiento.

Esto viene a sustituir a la forma tradicional de la carga de contenidos donde al cambiar de página se recarga todo el contenido de esta con la perdida de rendimiento que ello produce.

Una SPA funciona cargando el contenido HTML, CSS y Typescript (JavaScript compilado) por completo al abrir la web, al ir pasando de una sección a otra, solo necesita cargar el contenido que cambie nuevo, y no el resto de la página que ya está cargada.

Que sea una única página no implica que tenga todo el contenido cargado en una sección y que no exista distintas vistas. Lo que realmente hace Angular es cargar un módulo que gestiona toda la web y un contenedor principal que se encarga de cargar el contenido dinámico dentro del mismo, haciendo que cambie el aspecto y contenido de la página.

Es un framework modular que se basa en la creación de webs a través de componentes. Este concepto es muy práctico ya que permite crear distintos componentes dentro de una web, que realicen las funciones que necesitemos y la unión de dichos componentes me permiten crear aplicaciones web complejas.

El concepto de desarrollo por componentes es la base para crear aplicaciones web robustas y escalables, así como nos permite un desarrollo más ágil ya que podemos reutilizar los componentes que desarrollamos en varios proyectos.

Las aplicaciones de Angular solían ser modulares hasta la versión 17, a partir de su propio sistema, denominado NgModules.

Por defecto, toda aplicación Angular tenía como mínimo un módulo, el módulo raíz (**root**), conocido normalmente como AppModule.

Una aplicación pequeña lo normal es que estuviera constituida por un único módulo, pero lo habitual es que las aplicaciones más grandes se dividan en diferentes módulos. Los **Módulos** de Angular se representan a partir del decorador @NgModule Este decorador recibe la información de los metadatos que identifican el módulo con el que vamos a trabajar.

Las propiedades más importantes son las siguientes:

Declarations: especifica las vistas contenidas dentro del módulo en cuestión. Tenemos 3 tipos de vistas diferentes: **componentes**, **pipes** y **directivas.**
Exports: definimos aquellos componentes que serán visibles desde otros módulos externos.
Imports: nos permite especificar qué módulos externos vamos a necesitar dentro del módulo que estamos desarrollando.
Providers: lista de los servicios generados dentro de este módulo y que serán accesibles desde cualquier parte de la aplicación.
Bootstrap: define la vista principal de la aplicación. Sería lo que podemos denominar el componente principal

Un módulo puede agrupar diversos componentes relacionados entre sí.

Un módulo puede agrupar diversos componentes relacionados entre sí.

Los **componentes,** son lo que en programación denominamos las vistas. Como casi todo en angular son clases que definen métodos y propiedades que usa la propia vista para mostrar un resultado por pantalla.

Pueden tener atributos tanto de entrada (`decorador @Input`) como de salida (`decorador @Output`). También podemos definir al componente como:

Componente = template + metadatos + clase

Los **templates** definen la vista de un componente, el html y el css que va a usar el componente para mostrar el resultado.

Los **decoradores** son funciones que se colocan delante de la definición de una propiedad, clase a función y que ofrecen la posibilidad de configurar el comportamiento de esa clase, propiedad o función a la que decoran, son los **metadatos** de los componentes.

El **Data Binding** nos permite pintar datos o intercambiar datos con el template y la clase del componente, es la forma en la que comunicamos la parte de la lógica con la parte de la vista y podemos mostrar el resultado.

4.3. Standalone components vs módulos

En Angular 17, los **componentes independientes**, también conocidos como **standalone components**, se consolidan como una opción predeterminada, lo que representa un cambio importante en la forma de construir aplicaciones modulares. Esta nueva funcionalidad simplifica la estructura de las aplicaciones al permitir que los componentes, directivas y pipes se definan de manera autónoma, sin la necesidad

de estar incluidos en un módulo específico como antes ocurría con los módulos de Angular.

¿Qué son los componentes "standalone"?

Los **componentes standalone** son aquellos que pueden existir de forma independiente, sin estar declarados en ningún módulo. Esto significa que un componente puede funcionar sin necesidad de ser declarado dentro del tradicional NgModule, eliminando la dependencia de los módulos para el registro de componentes, directivas o pipes.

En versiones anteriores, los componentes siempre necesitaban ser declarados en un módulo, normalmente en AppModule u otros módulos secundarios. Con Angular 17, los componentes standalone facilitan la reutilización y el mantenimiento, ya que cada uno puede ser completamente autónomo y tener control total sobre sus dependencias.

Características de los componentes standalone:

1. **Declaración directa de dependencias**: Un componente standalone puede importar directamente otros módulos o dependencias sin necesidad de declararse en un NgModule. Por ejemplo, se pueden importar módulos como FormsModule o CommonModule dentro del propio componente.
2. **Más modularidad y reutilización**: Al no depender de los módulos, los componentes standalone fomentan la reutilización en diferentes partes de la aplicación o incluso en diferentes proyectos. Esto se traduce en una mayor flexibilidad y facilidad para compartir componentes entre aplicaciones.
3. **Optimización de la estructura**: Las aplicaciones con componentes standalone tienen una estructura más ligera y simplificada. Esto es especialmente útil en aplicaciones más pequeñas, donde crear un módulo completo para un conjunto de componentes puede ser excesivo.
4. **Simplificación en la creación**: Al usar la CLI de Angular (ng generate component), los componentes standalone ahora se generan

automáticamente sin necesidad de modificar módulos manualmente, ahorrando tiempo y esfuerzo.

La introducción de los componentes standalone representa un paso importante hacia un desarrollo más **modular y escalable**. Aunque los módulos siguen existiendo y son útiles para organizar aplicaciones grandes, los componentes standalone permiten una mayor flexibilidad, facilitando la construcción de **componentes más autónomos**, especialmente en casos de uso donde la modularidad completa no es estrictamente necesaria.

En resumen, Angular 17 transforma el enfoque hacia los componentes independientes, brindando a los desarrolladores más libertad para definir, reutilizar y organizar su código, lo que facilita la escalabilidad y el mantenimiento de aplicaciones a largo plazo

4.4. ¿Qué es un componente?

Una de las máximas en Angular es que **todo es un componente**. Representan la manera que tenemos en Angular para definir las unidades que vamos a visualizar en nuestras páginas y que contendrán la lógica de estas.

Por norma general, nuestras aplicaciones Angular estarán conformadas por un **árbol de componentes** relacionados entre ellos. Según el equipo de desarrollo de Angular:

«Un componente controla una parte de la pantalla llamada vista y se encarga de definir los diferentes bloques reutilizables por la interfaz de usuario de nuestras aplicaciones».

El objetivo principal que queremos cubrir en este tema es que entendamos cual es la importancia del desarrollo de componentes en Angular y como plantearnos la creación de un sistema basado en componentes.

Un componente es una clase a la que le precede un decorador @Component y que determina que ficheros conforman el componente. Dicho decorador suele determinar que un componente tiene una etiqueta que nos sirve para activar el componente dentro de otro o del componente principal.

Básicamente los componentes en Angular se organizan en una estructura de árbol teniendo un componente principal del cual depende todos los demás.
El trabajo con componentes en Angular dispone de dos partes fundamentales:

La definición del componente como elemento html. En este momento queda definido cómo vamos a acceder a dicho componente, la plantilla que lo representa y demás elementos relacionados con su aspecto.

El desarrollo de la lógica del componente. Todos los componentes llevan asociada una clase donde vamos a encontrar las acciones que van a poder llevar a cabo cada uno de los elementos declarados dentro de nuestro Componente.

4.5. Cómo crear y partes de un componente

Antes de ponernos a definir las partes de un componente veamos un componente tipo para poder hacernos una idea de cada una de sus partes.

```
import { Component, OnInit } from '@angular/core';

@Component({
  selector: 'app-primer-componente',
  templateUrl: './componente1.component.html',
  styleUrls: ['./componente1.component.css']
})
export class Componente1Component implements OnInit {

  constructor() { }
```

```
  ngOnInit(): void {
  }

}
```

Las primeras sentencias de nuestra clase definen todos aquellos import que nos permiten acceder a aquellas librerías que vamos a utilizar dentro de la clase.

Según vayamos conociendo o creando nuevas clases relacionadas con Angular y necesitemos usarlas dentro de nuestras clases, debemos agregar los import correspondientes.

En este caso únicamente estamos importando la librería @angular/core, donde encontramos la base para la creación de componentes en Angular

Por ejemplo, es donde se encuentra definido el decorador @Component**.** Mediante el decorador @Component especificamos los metadatos que van a caracterizar este componente.

El decorador se debe aplicar sobre la clase donde posteriormente implementaremos las acciones de nuestro componente.

Un componente se puede componer de tres partes:

¿Qué son las anotaciones? son los metadatos que agregamos a nuestro código y que permite definir ciertas propiedades, las más importantes son:

- **Selector:** Se trata de la propiedad que define cómo vamos a hacer referencia a este componente dentro del DOM de nuestra página. El nombre debe ir en minúsculas y separado por guiones en caso de que contenga más de una palabra. Cuando queramos utilizar nuestro componente, lo especificaremos de la siguiente manera:

```
<app-primer-componente></app-primer-componente>
```

template / templateUrl: Con estas propiedades especificamos el contenido que va a renderizar nuestro componente cuando en el DOM situemos el selector que lo define. El primero, define el contenido de manera INLINE, mientras que el segundo nos permite concretar un fichero .html donde encontraremos la plantilla que vamos a renderizar.

- styles / styleUrls: Podemos definir los estilos asociados a nuestro componente a través de estas dos propiedades. Mediante la propiedad style podemos definir los estilos de nuestro componente en línea. Debemos trabajar como si estuviésemos generando los estilos para cualquier página, es decir, establecer clases, modificar elementos a través de su identificador o a través de su tag.

```
import { Component } from '@angular/core';
@Component({
  selector: 'app-primer-componente',
  template: `<h1>Hola Angular 2</h1>`,
  styles: [`
      h1 { color: #fff; }
      .description {
      font-style: italic;
      color: green;
      }
 `]
})
export class AppComponent { }
```

Podemos incluso dividir los diferentes estilos en cadenas de caracteres contenidas dentro del array de styles.

La parte negativa de usar esta nomenclatura es que la definición de los parámetros de nuestro componente se vuelve demasiado compleja e ilegible, lo que entorpece las posibles tareas de mejora y escalabilidad.

Mediante la propiedad styleUrls podemos permitirnos definir los estilos asociados a un componente en un fichero .css externo.

Al ser un array de literales, nos permite definir tantas hojas de estilo como sean necesarias para nuestros componentes.

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  title = 'UnirApp';
}
```

Aparte de las dos propiedades de estilos que tenemos disponibles en la definición de los metadatos de `@Component`, podemos definir los estilos de nuestros componentes de dos maneras diferentes.

La primera sería agregando la etiqueta `<style>` delante del html que vamos a renderizar con el componente.

De igual manera, podemos asignar el atributo style a cualquiera de los elementos que coloquemos dentro de nuestra plantilla.

Como crear un componente

Generalmente, un componente está formado por cuatro archivos, aunque uno de ellos no es obligatorio y se puede omitir.

- app.component.html: Es la vista que contiene el HTML del componente y donde se van a realizar otras las funcionalidades de renderizado.

- app.component.css: Define la hoja de estilos de un componente. Nos permite repetir otras etiquetas que hayamos usado dentro de otros componentes ya que angular les añade una capa numérica a las hojas de estilos y así no se repiten.

- app.component.ts: Esta escrito en TypeScript y es el **controlador** de nuestra aplicación. Es donde está definido toda la lógica de negocio de nuestro componente y donde implementaremos todos los métodos necesarios para que nuestro componente funcione.

- app.component.spec.ts: los ficheros spec.ts nos sirven para hacer test unitarios y que nuestro resultado sea más fiable.

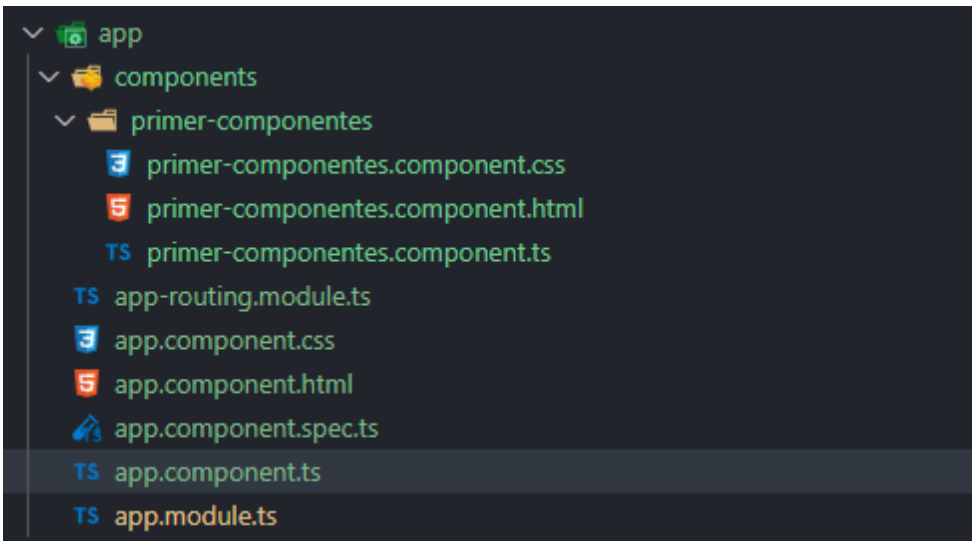


Figura 1. Árbol de carpetas de un componente. Fuente: contenidos MFPFSTDEV.

Para crear un componente nos vamos a servir de la herramienta angular CLI que instalamos al principio de este tema y que entre otras muchas cosas nos va a servir para crear componentes y otros elementos de una aplicación en angular.

Para crear un componente escribimos en la terminal elegida el comando:

```
ng generate component components/nombreComponente –skip-tests
```

También tienes un formato corto para este script en terminal:

```
ng g c components/nombreComponente
```

El skip-tests sirve para evitar la creación del fichero de test en cada component usará siempre que quieres que dicho fichero no aparezca en la carpeta del componente.

4.6. Componentes principal y componentes hijos. ¿Cómo se enlazan?

Una de las herramientas más potentes dentro de Angular es la posibilidad de **anidar componentes.**

Para crear una aplicación compleja, podemos implementar los componentes de la manera más sencilla posible para posteriormente, combinarlos entre sí.

Si queremos utilizar un componente dentro de la estructura html de otro, lo podemos hacer incluyendo el selector del componente hijo dentro del espacio donde queramos que se renderice del componente padre

Para que esto sea posible, no debemos olvidar **declarar nuestros componentes dentro del módulo donde estemos trabajando.**

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';

import { AppRoutingModule } from './app-routing.module';
import { AppComponent } from './app.component';

//aquí importamos el componente
import { PrimerComponentesComponent } from './components/primer-componentes/primer-componentes.component';
```

```
@NgModule({
  declarations: [
    AppComponent,
    // importantisimo añadirlo al array de declarations, esto lo hace por
nosotros angular cli pero debemos comprobarlo.
    PrimerComponentesComponent
  ],
  imports: [
    BrowserModule,
    AppRoutingModule
  ],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

Por último, utilizamos el componente hijo dentro de la plantilla del padre. Recordamos que ahora mismo solo tenemos un componente padre que es app.component.ts**.** Pero esto es extrapolable a otros componentes no solo al componente principal.

Enlazar ambos componentes tenemos que poner la etiqueta del componente hijo dentro del fichero .html del padre. De esta forma:



Figura 2. Cómo se enlazan dos componentes. Fuente: contenidos MFPFSTDEV

Esta forma de enlazar componentes dentro de otros componentes. Si ahora intentamos levantar nuestra aplicación para ver el resultado, solo tenemos que hacer el siguiente comando en la terminal.

```
ng serve -o
```

Esto arrancará nuestra terminal y el resultado que veremos será el siguiente, ya que dentro de nuestro componente principal habremos cargada el componente primer-componente visualizándose de la siguiente manera.

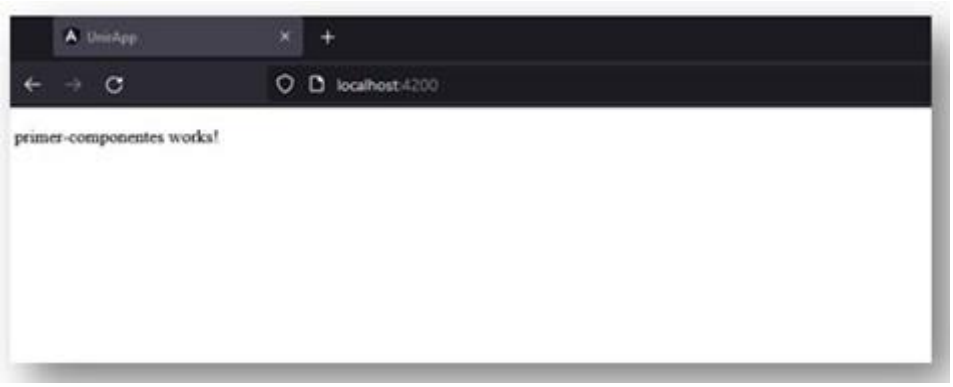


Figura 3. Resultado de la carga del primer componente. Fuente: contenidos MFPFSTDEV

Si modificamos algo del componente, por ejemplo, poner un título en el html del propio componente, de esta forma:

```
<h1>Titulo del primer componente</h1>
<p>Primer parrafo</p>
<p>Segundo parrafo</p>
```

El resultado que saldrá por pantalla tendrá la siguiente pinta:

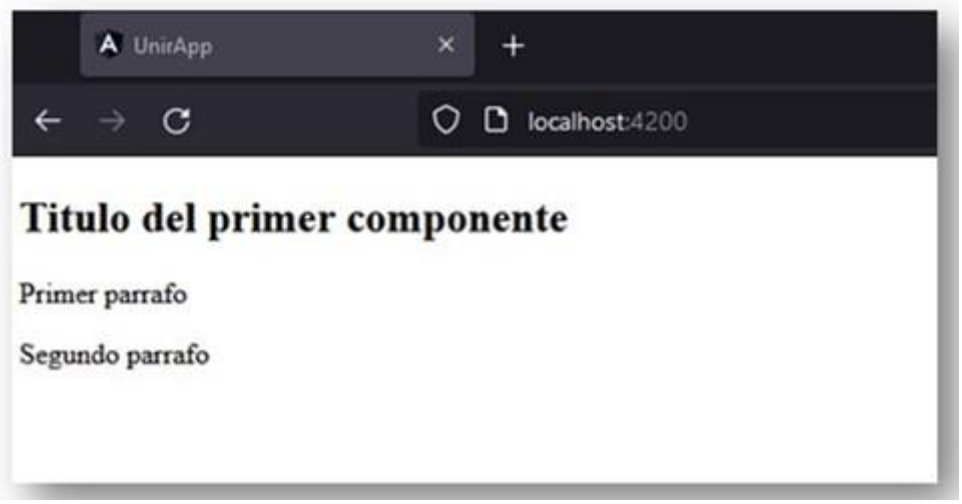


Figura 4. Resultado del navegador. Fuente: contenidos MFPFSTDEV

Simplemente hemos incluido algo de maquetación en html dentro del componente sin ninguna funcionalidad, pero el resultado igualmente se visualiza en pantalla.
En el siguiente tema veremos como enlazar el html y css de cada componente a través de su sistema de templating.

4.7. Funciones de ciclo de vida de un componente

Todos los componentes de Angular disponen de una serie de métodos que podemos implementar y que se ejecutan automáticamente en determinados momentos del ciclo de vida del componente, dándonos la posibilidad de interactuar con cada uno de estos momentos.

De esta manera seremos capaces de realizar diferentes acciones desde la creación del componente hasta su destrucción.

El siguiente gráfico muestra el orden de ejecución de los diferentes métodos del ciclo de vida de un componente.

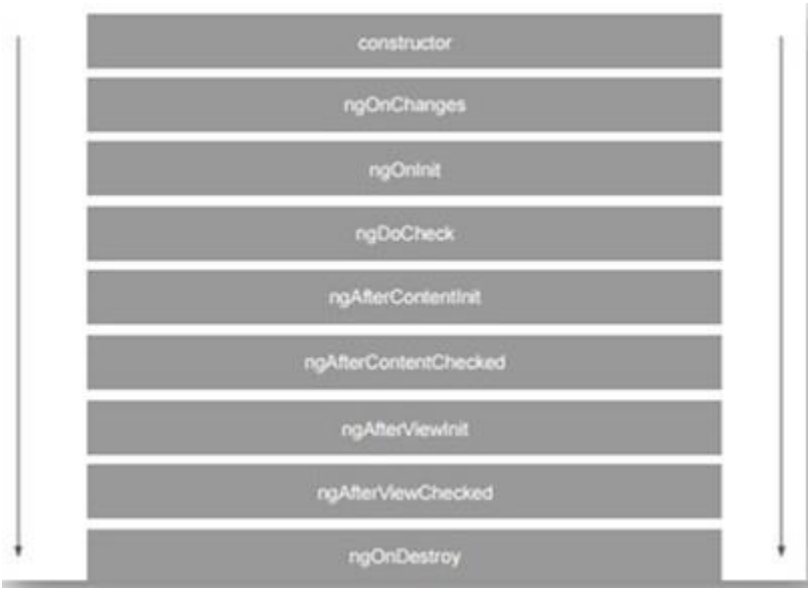


Figura 5. Orden de ejecución del ciclo de vida de un componente. Fuente: contenidos MFPFSTDEV

Todas ellas tienen un tiempo de ejecución al que podemos engancharnos para ejecutar algunas de nuestras acciones dentro del componente, algunas incluso se solapan y pueden llegar a confundirse por que se ejecutan al mismo tiempo.

Vamos a hacer un repaso por las más importantes y que vamos a usar la mayoría de las veces en nuestros desarrollos.

Constructor

Es el método que carga la lógica y he inicializa las propiedades del componente.

ngOnChanges

Este método es llamado cuando se aprecia algún cambio en cualquiera de las propiedades que recibe nuestro componente. Vemos un pequeño ejemplo de implementación.

```
ngOnChanges(changes: SimpleChange) {
   for (let propName in changes) {
     let chng: any = changes[propName];
     let cur = JSON.stringify(chng.currentValue);
     let prev = JSON.stringify(chng.previousValue);
```

```
        console.log(`${propName}: ValorActual = ${cur}, ValorPrevio =
${prev}`);
    }
  }
```

Estos cambios se aprecian siempre y cuando modifiquemos el valor que le estamos pasando al componente desde el elemento donde lo tenemos definido.

Estos cambios únicamente se observan en un primer nivel. Es decir, si estamos recibiendo un objeto como valor de un input, las modificaciones que se realicen dentro de los atributos de ese objeto no serán detectadas por el método.

`ngOnInit`

Ocurre tras la primera ejecución de `ngOnChanges`. Es un método que solo ocurre una vez cuando el componente se carga por primera vez. Es un método que esta implementado en el interfaz OnInit y que inicializa no solo la lógica del componente si no también la parte del HTML.

Implementamos este método por dos razones:

- Para realizar acciones complejas justo después del constructor.
- Para configurar el componente después de haber definido los inputs de este.

En el constructor no se deben hacer tareas costosas como la obtención de datos para el componente o la llamada a servicios. Este método sería el lugar idóneo para llevar a cabo estas tareas.

Este método se ejecuta prácticamente a continuación de la creación del componente.

`ngDoCheck`

Hemos visto cómo podemos utilizar el método `ngOnChanges` para recibir los cambios en las diferentes propiedades de nuestros componentes. Estos cambios no se reflejan si las propiedades con las que trabajamos con objetos complejos o arrays. Para ello

podemos implementar el método `ngDoCheck`, el cual nos permite trabajar con una comprobación personalizada de los datos. Este método se lanza después de detectar cualquier tipo de cambio en los datos de nuestro componente. No se debe abusar del uso de este método ya que puede afectar al rendimiento de nuestra aplicación

`ngAfterContentInit`

Se lanza **después de inicializar el contenido del componente**.

`ngAfterContentChecked`

Se ejecuta **tras cada comprobación del contenido del componente**. Comprueba el contenido visualizado en el componente.

`ngAfterViewInit`

Se ejecuta **después de que las vistas del componente se inicialicen** y después de ngAfterContentChecked.

`ngAfterViewChecked`

Se ejecuta **después de cada comprobación de la vista de un componente**. Se llama después de ngAfterViewInit y de cada ngAfterContentChecked.

`ngOnDestroy`

Se trata del método donde colocaremos las **acciones de limpieza de nuestro componente.**

En este momento deberíamos avisar al resto de componentes, si fuera preciso, que el componente actual va a desaparecer. También es el lugar adecuado para liberar recursos utilizados durante el ciclo de vida del componente.

Hay que tener en cuenta las posibles fugas de memoria que pueda provocar nuestro componente.

4.8. Hojas de estilos de componente

A la hora de trabajar con estilos de CSS Angular también nos aporta soluciones bastante interesantes.

La primera es que al tener una hoja de estilos por componente los estilos de este son independientes de otros componentes, lo cual hace que no nos tengamos que preocupar del nombre de las etiquetas de otro componente porque Angular las hace independientes.

Si queremos que algún estilo afecte a todos los componentes tenemos la hoja de estilos principal style.css.

En esta hoja de estilos podemos cargar los estilos que van a ser comunes y que queremos que afecten a todos los componentes.

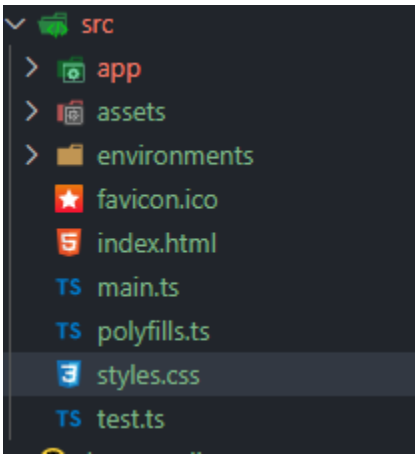


Figura 6. Style.css. Fuente: contenidos MFPFSTDEV

Por el contrario, si usamos las hojas de estilos propias de cada componente, da igual que estos sean hijos de otros, dichos estilos solo afectaran al componente al que pertenezca esa hoja de estilos, y exclusivamente a él.

Esto hace que no tengamos que estar pensando en si ese nombre de clase que hemos puesto ya lo hemos usado en otro componente y sobre todo permite que si dos

desarrolladores trabajan en distintos componentes y usan clases iguales su resultado no se vea afectado por el desarrollo del otro.

Uso de Sass

Dentro de Angular también podemos usar Sass como preprocesador de CSS. Solo hay que tener en cuenta ciertos parámetros.

El primero es que cuando creamos el proyecto debemos marcar que este va a ser con SASS o con SCSS según preferencia. Al elegir una de estas opciones nuestras hojas de estilo CSS desaparecerán para dejar paso a SCSS tanto como hojas generales como dentro de los componentes.

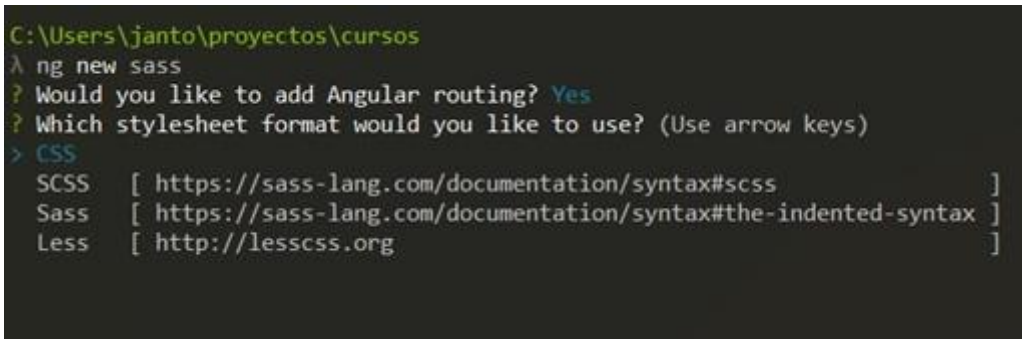


Figura 7. Menú de elección de SCSS. Fuente: contenidos MFPFSTDEV

El propio compilador de Angular se encargará de transformarlas a CSS y que podemos ver el resultado en tiempo real dentro de nuestro localhost.

Otro punto importante a la hora de trabajar con SCSS en Angular es que tenemos que configurar el angular.json para decirle donde se van a alojar los ficheros partials de SCSS dentro de nuestra aplicación. Lo normal para que estén accesibles para todos los componentes que los importen es en la carpeta assets, de la siguiente forma. Damos de alta justo debajo de script una propiedad stylePreprocessorOptions que contiene otro propiedad donde se incluyen la ruta donde van a ser alojados los archivos partials de CSS, esos que empiezan por guión bajo _variables.scss, etc.

```
"styles": [
```

```
        "src/styles.css"
    ],
    "scripts": [],
    "stylePreprocessorOptions": {
          "includePaths": ["assets/partials"]
     }
```

A partir de colocar dentro de esa carpeta todos los archivos parciales de angular, cada componente los podrá importar de forma personal el que necesite en cada momento y de forma independiente.

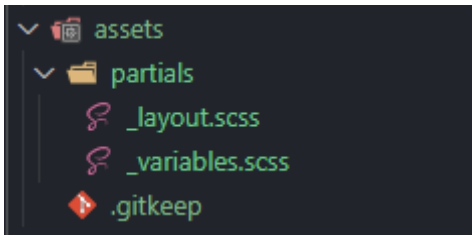


Figura 8. Sistema de carpetas SCSS dentro de un proyecto. Fuente: contenidos MFPFSTDEV

De esta forma podemos dar estilos a nuestros componentes ya sea a través de CSS puro y duro como a través de SASS, trabajando de forma individual con cada componente cuando sea necesario y de forma genérica a través de la hoja principal ya sea style.css o style.scss.

El uso de cualquiera de estas tecnologías de estilo dependerá del equipo de trabajo ya que son tecnologías muy parejas pero que ofrecen ventajas e inconvenientes a la hora de maquetar. Conocerlas y usarlas bien es fundamental para obtener un resultado óptimo visualmente en nuestros desarrollos.

4.9. Referencias bibliográficas

Aristeidis Bampakos, P. D. (2023). *Learning Angular: A no-nonsense guide to building web applications with Angular 15.* Packt Publishing.

Bampakos, A. (2023). *Angular Projects: Build modern web apps in Angular 16 with 10 different projects and cutting-edge technologies.* Packt Publishing.

Google. (11 de 07 de 2024). *Android Developers*. Obtenido de https://developer.android.com/

Microsoft. (15 de 07 de 2024). *TypeScript official web site*. Obtenido de https://www.typescriptlang.org/

Uluca, D. (2024). *Angular for Enterprise Applications - Build scalable Angular apps using the minimalist Router-first architecture.* Packt Publishing.

A fondo

Manual de Angular

Desarrollo web. https://desarrolloweb.com/manuales/manual-angular-2.html

Monográfico de angular en español donde podréis afianzar concepto.

Componentes en Angular

Ro. (31 de julio de 2019). *Entendiendo los componentes en Angular - Guía de iniciación* [Blog]. https://www.acontracorrientech.com/entendiendo-los-componentes-en-angular/

Un blog interesante donde se explican cómo funcionan los componentes en angular.

Cómo usar los StandAlone Components en Angular

Garaje de ideas | Tech https://www.youtube.com/watch?v=DioXD75_t-Q

Video analizando como usar los StandAlone components en Angular.